Memory-Aware Agent Edge Serving: Phase 1 Baseline and Measurements

Yunxuan Li [1] **Max Huang** [1]

Abstract

On a memory-constrained machine, how should a server admit and schedule concurrent agent tasks so that completed work increases while memory stays bounded? We build a runner-owned FIFO baseline over llama.cpp with a seeded workload of short, long-context, long-generation, and multi-step agent tasks, and measure 45 runs (17.9 h) on a CPU-only laptop. Memory is fixed at launch, so the budget is KV capacity. A static slot partition forces a choice between concurrency and admissible context, and the dominant bottleneck is CPU compute surfacing as FIFO queueing. Retained context helps but is fragile. These findings motivate KV-budget admission with size-aware ordering for Phase 2.

1. Problem and Setup

Question. Admitting too much agent work raises latency or exhausts memory; admitting too little idles the machine. Phase 1 builds the FIFO baseline and measures what limits it.

Environment (all pinned). MacBook Air (M4, 16 GB, 10 cores) on AC power, no thermal warning recorded. CPU-only `llama-server` (llama.cpp `1b29621`, `b11741`, Metal off), Llama-3.2-3B-Instruct Q4_K_M (sha256 `6c1a2b41…28ff`), `-t 8 --cache-ram 0`. The memory budget B is the total KV-token count (`-c 16384`, 1792 MiB at 112 KiB/token). *Series S* fixes B and sets `-np`= c, so each slot holds B/c tokens; *series U* uses one shared pool (`-kvu -np 8`).

Workload. A seeded, byte-reproducible Dev workload (seeds 1–3 here; Eval seeds 101–105 with a different mix are untouched): 100 tasks, mix 40/20/20/20 of *short* (256 in/64 out tokens), *longctx* (3000/64), *longgen* (256/512), and *agent* (about 800-token context, 3–6 steps of 128 tokens, tool delay 0.2–3 s). Arrivals are open-loop Poisson. Each task embeds a random needle checked programmatically (single-stream pass rate 100/95/100/95%). The runner owns a step-level FIFO with $c \leq$ `-np`; tool delays hold no slot.

Metrics and rules. Per-request and per-task logs, a 0.5 s system sampler, and a per-run manifest (git sha, versions, command line, workload hash). Deadline is $5\,T_{\text{solo}}$ (frozen); task timeout is $\min(300, 10\,T_{\text{solo}})$ s with active disconnect. Failures and timeouts stay in every denominator; the first 3 tasks of each run are warm-up and excluded. Peak memory is macOS `footprint`, because RSS is unreliable here (idle RSS of one configuration ranged 689–4678 MB across launches); `footprint` omits about 294 MB of memory-mapped weights, so it is a lower bound.

2. Results

All 45 Dev runs passed validity checks: concurrency sweep E1 ($c \in \{1, 2, 4, 8\}$ in S, $\{4, 8\}$ in U, λ=0.088 task/s), tool delay E3 (0–3 s), `cache_prompt` on/off E4 ($c \in \{1, 4\}$), and arrival rate E5 (c=4), each with 3 seeds.

F1: Memory is fixed at launch, so the budget is KV capacity. The KV buffer is allocated and zeroed once at startup, and peak `footprint` is 3.84–3.90 GB in every run (Fig. 1c), independent of c, λ, and series. Doubling `-c` raised idle `footprint` by 1795 MB, matching the 1792 MiB KV difference. Memory pressure thus appears as slot capacity and admission, not as growth.

F2: A fixed partition trades concurrency against admissible context. At c=8 a slot holds 2048 tokens, so all 62 longctx tasks and 11 of 53 agent tasks fail within seconds with `error_ctx_exceeded` (Table 1). The c=8 goodput of 0.052 task/s (vs. 0.015 at c=4) comes from rejecting the heaviest task class, not from serving faster, so c=8 is *not* a valid baseline. Series U has no admission control and was not better than S at c=4 (65% vs. 59% failed, sd about 10 points). Source reading shows pool exhaustion returns HTTP 500 to *all* running requests; our 6 U runs never hit it. All c are overloaded at λ=0.088; excluding c=8, c=4 has

[1]CSE 291A (Fall 2026), UC San Diego; course project, Option 1, Phase 1 report. Correspondence to: Yunxuan Li <yul386@ucsd.edu>, Max Huang <mah067@ucsd.edu>.

Preprint. November 1, 2026.

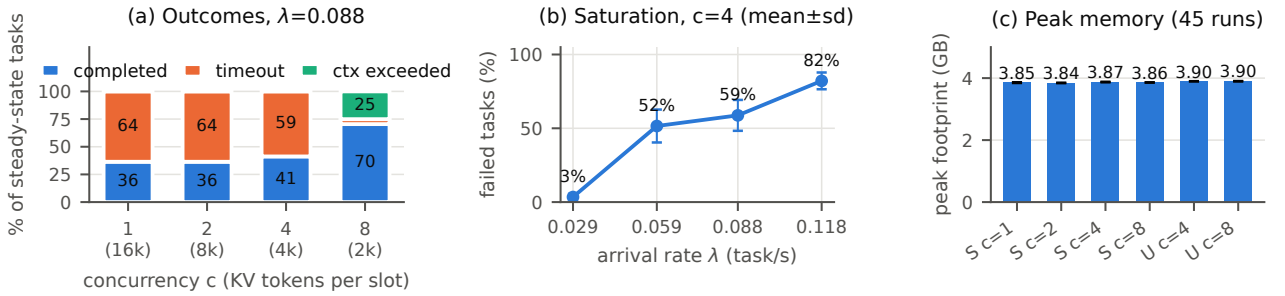


Figure 1. (a) Outcome of steady-state tasks (3 seeds, 291 tasks per bar). (b) Failure rate at c=4 versus arrival rate. (c) Peak `footprint` for every configuration (S: np=c; U: shared pool). Data: `results/analysis/batch/*.csv`.

Table 1. Completed tasks by type (S, λ=0.088, 3 seeds). All other failures are task timeouts unless marked "ctx".

Type (total)	c=1	c=2	c=4	c=8
short (116)	8	9	22	111
longctx (62)	62	62	62	0 (62 ctx)
longgen (60)	27	23	19	52
agent (53)	9	12	17	42 (11 ctx)

the best goodput and is our fixed-concurrency baseline. It also passes a relaxed rule (no `ctx_exceeded`, at most 10% failures) at λ=0.029, where c=1, 2 were not run. We relaxed the rule after seeing the data; the original p99≤deadline rule picks c=8.

F3: The dominant bottleneck is CPU compute, expressed as FIFO queueing. Aggregate throughput barely grows with concurrency: 38.3 tok/s at c=4 vs. 42.8 at c=8. Service time inflates under contention even when unsaturated (c=4, λ=0.029): median longgen 29.6 s vs. T_{solo} 15.95 s, and longctx 35.1 vs. 26.4 s. The capacity estimate μ=0.098 task/s was too high: at c=4 failures rise 3%→52%→59%→82% across λ=0.029 . . . 0.118 (Fig. 1b), so saturation lies near 0.03–0.05 task/s (0.3–0.5 μ). Above saturation almost all failures are timeouts, and short tasks suffer most: 81% timed out at c=4, with a median queue wait of 31.9 s against a 32 s limit, while 26–45 s longctx tasks hold slots. This is consistent with head-of-line blocking, which we have not yet isolated.

F4: Retained context matters but is fragile. With `cache_prompt` on, agent steps ≥ 2 reuse 24% (c=1) and 54% (c=4) of their prompt tokens, versus 0% off. Paired on/off differences favor caching (p50 −27 s at c=1, −36 s at c=4; failures −9.6/−18.6 points), but both sides are overloaded and there are only 3 pairs with large sd (14 and 51 s), so only the direction is reliable. We observed the mechanism directly: during a tool delay, an unrelated short request was routed by prefix similarity (0.125, above the 0.10 threshold) to the agent's slot and overwrote its prefix. E3 (tool delay 0–3 s) shows no trend; overload swamps it.

3. Phase 2 Direction

FIFO with a fixed c cannot see the one resource that differs across tasks: KV tokens. A static partition rejects long contexts at high c; a low c serializes everything behind long steps (F2, F3). We will build *one* mechanism: **KV-budget admission with size-aware ordering on the shared pool.** Each ready step has a known cost (prompt tokens plus `max_tokens`). The scheduler admits it only if the sum of reserved tokens stays within B, so concurrency adapts to task size (many short steps, few long ones) with no `ctx_exceeded` and no pool exhaustion. Among ready steps it prefers cheaper and continuing agent steps, relieving F3. A cache-aware extension pins a continuation to its previous slot via the request-level `id_slot` (present in the source; to be verified on the OAI endpoint) and is compared against the fixed policy (`cache_prompt` on) and off (F4).

Evaluation plan. Compare against FIFO and the tuned fixed-c baseline on the Eval workload (seeds 101–105, different mix), sweeping λ from under- to overload, including the underloaded case where no gain is expected. Report completed tasks, p50/p99 latency, throughput, peak memory, failure rate, and quality pass against the 100/95/100/95% baseline, so gains cannot come from skipped tasks or shortened output, plus policy overhead and long-task fairness (longctx p99). Timeline: Appendix A; full metrics: Appendix B.

4. Limitations

Single laptop, CPU-only, one model: numbers may not transfer to AWS or Metal. Three seeds of 100 tasks, and most E1/E3/E4 points are in overload, so effect sizes are weak; the λ grid came from a capacity estimate that proved too high. Background load (XProtect) was not excluded.

Appendix A. Timeline and Milestones

Phase 1 (completed; dates from the git history and lab notebook).

Date	Milestone
Oct 18	Option 1 chosen; course and project requirements read.
Oct 19	Sample reports reviewed; pinned llama.cpp and model; memory baseline; capacity calibration; metrics fixed. Runner built test-first against a fake server: workload generator, FIFO, Poisson arrivals, timeouts, agent tasks, sampler, manifest, analysis command.
Oct 30	Pre-batch checks: `-t 8` frozen after interleaved comparison; T_{solo} re-measured on AC power (23–29% faster, so λ grid and deadlines recomputed); c=8 throughput; cold start.
Oct 31	Real-server smoke test (30 tasks, logs complete); single-stream quality baseline and frozen deadlines (Dev workload hash fixed); resumable batch runner with 30-task pilots.
Oct 31–Nov 1	45-run E1–E5 batch (about 17 h, no failed runs or throttling); analysis; correction that c=8 looks best only because it rejects longctx. This report written Nov 1.

Phase 2 plan (due December 10, 2026).

Window	Milestone and exit criterion
Nov 2–8	**M1, baseline.** Confirm baseline c=4 by running E1 at λ=0.045 (c=1, 2, 4); verify `id_slot` pinning on the OAI endpoint; deliberately exhaust a U pool to observe the failure mode. *Exit:* baseline c confirmed in a non-overloaded regime.
Nov 9–15	**M2, mechanism.** Implement KV-budget admission and size-aware ordering behind the scheduling-policy interface; fake-server tests; 30-task pilots. *Exit:* no admitted step exceeds the budget.
Nov 16–22	**M3, tune and freeze** on Dev seeds only; ordering ablation to test the head-of-line explanation; freeze parameters.
Nov 23–Dec 1	**M4, Eval runs** (seeds 101–105; λ sweep and mix shift; FIFO vs. tuned fixed c vs. new policy; cache-aware vs. fixed cache policy; at least 3 repeats). Thanksgiving, Nov 26: runs unattended.
Dec 2–6	**M5, analysis.** Quality parity, overhead, fairness, no-benefit regimes, failure accounting.
Dec 7–10	**M6, report and artifacts.** Final report; clean-clone reproduction of all figures; tag and submit on Dec 10.

Appendix B. Performance Summary

Mean over 3 seeds (Dev workload, warm-up excluded; `cache_prompt` on except E4). p50/p99 cover *completed* tasks only, so they flatter configurations that reject work (S, c=8 rejects all longctx). Thr. = completed tasks/s over the steady-state window; Goodput counts only tasks meeting deadline and quality check; Fail = share of tasks not completed; Peak = max `footprint`. Standard deviations and queue times: `perf_by_c_macos.csv`.

Exp.	Configuration	p50 (s)	p99 (s)	Thr.	Goodput	Fail (%)	Peak (GB)
E1	S, c=1, λ=0.088	153	242	0.028	0.005	64	3.85
E1	S, c=2, λ=0.088	143	247	0.028	0.006	64	3.84
E1	S, c=4, λ=0.088	102	254	0.033	0.015	59	3.87
E1	S, c=8, λ=0.088	18	142	0.063	0.052	30	3.86
E1	U, c=4, λ=0.088	122	265	0.027	0.010	65	3.90
E1	U, c=8, λ=0.088	51	210	0.020	0.013	76	3.90
E4	S, c=1, cache off	180	257	0.020	0.002	73	3.85
E4	S, c=4, cache off	138	258	0.017	0.004	77	3.87
E5	S, c=4, λ=0.029	25	170	0.029	0.025	3	3.87
E5	S, c=4, λ=0.059	74	275	0.028	0.017	52	3.87
E5	S, c=4, λ=0.118	177	261	0.017	0.003	82	3.87
E3	S, c=4, delay 0 s	103	242	0.031	0.014	61	3.87
E3	S, c=4, delay 0.5 s	105	233	0.031	0.013	61	3.87
E3	S, c=4, delay 1 s	114	248	0.031	0.012	60	3.87
E3	S, c=4, delay 3 s	112	237	0.029	0.013	63	3.87

Reproduction. Raw runs: `results/raw/batch/`. `python -m src.analysis` regenerates the CSVs; `report/phase1/make_figs.py` and `make_table.py` regenerate Fig. 1 and this table.